

CSCI 12532

Computer Architecture and Design

Lecture 01

Ms. Meesha Mervyn
Dept. of Computer Systems Engineering
Faculty of Computing and Technology
University of Kelaniya
meesham@kln.ac.lk

Learning Outcomes of the Module

A student who successfully completes this course will be able to,

LO1: Understand the basic architecture of a computer.

LO2: Describe how computer executes instructions.

LO3: Implement simple assembly programs

Assessment Strategy

Continuous Assessments:

- Group Assignment - 10%
- Mid Exam (7th Week) - 25%
- Pop Quizzes - 5%

Final Evaluation

- Theory Examination - 60%

Contents

1. What is Computer Architecture?
2. Introduction to LMC
3. Components of the Little Man Computer
4. Registers
5. Instruction Set
6. Execution Cycle
7. Memory
8. Limitations
9. Sample Program

1. What is computer architecture?

Imagine that you want to build a restaurant. What type of planning does it require?

First you must specify

- The structure of the restaurant
- The operations it offers
- How the components interact

This specification is called the ARCHITECTURE.

Then you must also design

- The layout of the restaurant and kitchen
- Type of equipment
- The staff hierarchy

These implementation details are called the ORGANIZATION

Computer Architecture

The architecture defines those attributes of the system apparent to the programmer or those attributes that directly force the program's logical execution. It manages with the functional behavior of the computer system view by the user.

Computer Architecture manages with the system's programming part, including the multiple bits can define the several data types, input-output methods, approaches for addressing memory, and the computer's instruction set. The implementation of any program is influenced by these qualities of architecture. The change in a bit or size can lead to a different result for the programmer.

You can have the same computer architecture (e.g., an app written for a specific instruction set) running on billions of devices, but the computer organization in an Apple Watch, a smartphone, and a smart TV is vastly different, optimizing for power, performance, and cost respectively.

The main pillars of Computer Architecture

- System design: This is what makes up the structure of a computer, including all hardware parts, such as CPU, data processors, multiprocessors, memory controllers, and direct memory access.
- Instruction set architecture (ISA): This is any software that makes a computer run, including the CPU's functions and capabilities, programming languages, data formats, processor register types, and instructions used by programmers.
- Microarchitecture: This defines the data processing and storage element or data paths. These include storage devices and related computer organisation tools.

2. Introduction to Little Man Computer

The Little Man Computer - an instructional model of von Neumann computer architecture.

The Little Man Computer (LMC) is a simplified computer model designed to teach the basics of computer architecture and programming. It is a great tool for beginners who want to learn how computers work and how to write simple programs.

- Simplified Computer Model: Designed to teach the basics of computer architecture and programming.
- Beginner-Friendly: Ideal for learning how computers work and how to write simple programs.
- Minimalist Design: Uses a few registers, small memory, and a limited instruction set to simplify understanding.
- Von Neumann Architecture: Simulates modern computer architecture.
- Invented in 1965: Created by Dr. Stuart Madnick and still widely used as a teaching tool.
- Little Man Concept: Represents the control unit of a CPU, handling fetching, decoding, execution, and I/O management.

Instruction Cycle of the LMC (Fetch-Decode-Execute)

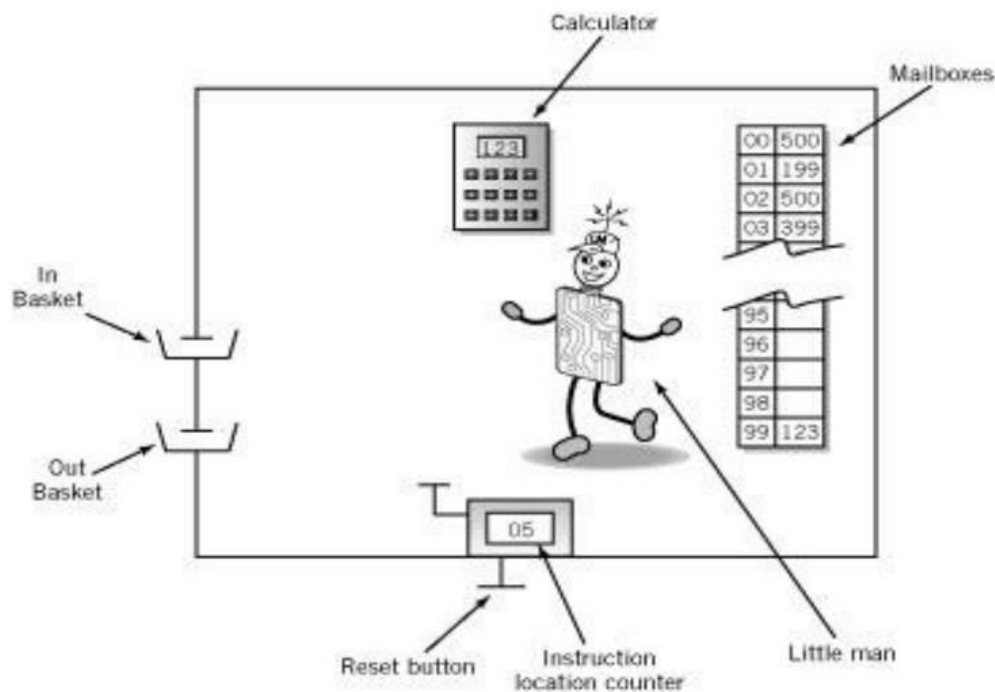
This cycle, which is similar for all the instructions, can be broken into two parts.

1. The Fetch Portion of the cycle, in which the Little Man finds out what instruction he is to execute, and
2. The execute portion of the cycle, in which he actually performs the work specified in the instruction

The fetch portion of the cycle has to occur first: until the Little Man has performed the fetch operation, he does not even know what instruction he will be executing.

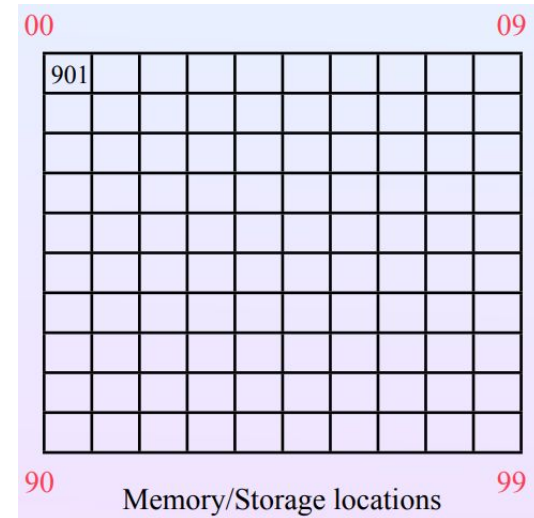
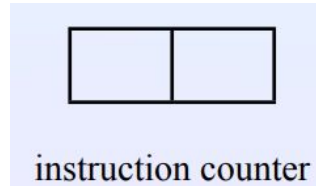
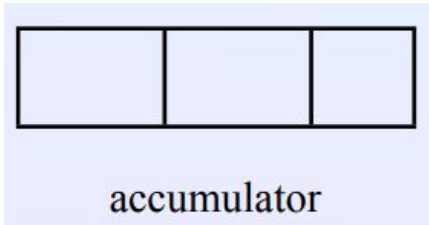
The execute portion of each instruction is, of course, different for each instruction.

The Little Man Computer



3. Components of the Little Man Computer

- 100 storage locations indexed 0 thru 99 - each can store a 3 digit integer
- a unique 3 digit storage location called calculator or accumulator
- a 2 digit instruction location counter/program counter
- an inbox that can contain a 3 digit number
- an outbox that can contain a 3 digit number



4. Registers

Registers are a Small, high-speed memory locations used by the CPU to store data being processed. There are two Main Registers in LMC: Accumulator and Program Counter.

Accumulator

- - Holds data being manipulated or processed by the CPU.
- - Stores positive or negative values.
- - Modified using arithmetic operations like ADD and SUB.
- - Stores the results of operations.

Program Counter (PC)

- - Keeps track of the address of the instruction to be executed.
- - Fetches instructions from memory.
- - Increments to the next instruction after execution.

5. LMC Instruction Set

Instruction	Mnemonic	Machine Code	Further Information
Load	LDA	5xx	Load the contents of address xx onto the accumulator. <i>Note: the contents of the address are not changed.</i>
Store	STA	3xx	Store the contents of the accumulator to address xx. <i>Note: the contents of the accumulator are not changed.</i>
Add	ADD	1xx	Add the contents of address xx to the accumulator. <i>Note: the contents of the address are not changed and the total cannot exceed 999.</i>
Subtract	SUB	2xx	Subtract the contents address xx from the accumulator. <i>Note: the contents of the address are not changed.</i> <i>Note: If a subtract instruction causes negative results then a negative flag will be set so that BRP can be used properly.</i>
Input	INP	901	Copy the value from the “in box” onto the accumulator.
Output	OUT	902	Copy the value from the accumulator to the “out box”. <i>Note: the contents of the accumulator are not changed.</i>
End	HLT	000	Stop the LMC simulator executing the program.
Branch always	BRA	6xx	Stop the LMC simulator executing the program.
Branch if zero	BRZ	7xx	If the contents of the accumulator are ZERO , set the program counter to address xx.
Branch if zero or positive	BRP	8xx	If the contents of the accumulator are ZERO or positive (i.e. the negative flag is not set), set the program counter to address xx.
Data storage	DAT		Reserve as data the memory address reached when this instruction is compiled. <i>Note: a value N can be stored at the memory address by using DAT N</i>

A program in LMC

Consists of instructions placed in memory starting at position 00.

An instruction is a 3 digit integer

- left digit an operation code 0 to 9 - telling what type of action to take right 2 digits 0 to 99 indicate memory position

Instructions are of various sorts :

- some take information from inbox and place in the accumulator - such information called input
- some take information in the accumulator and place in outbox - such information called output
- some interact with information in accumulator

Construction of Little Man Program

The source code of a little man program is a list of mnemonic instructions.

- The collection of mnemonic instructions constitute the assembly language for Little Man programs
- The source code is compiled by a background program called the compiler
- The compiler places equivalent numeric 3 digit instruction codes sequentially in memory

Running an LMC program

Once compiled (an operation external to the program), the program is started by another operation external to the program called run

- Once started the first instruction in location 00 is executed and the instruction location counter is incremented from 0 to 1
- The first Little Man instruction is often (but not always) an Input instruction - an outside agent puts information in the inbox for the input instruction to execute

Instructions

Commands that tell the computer to perform specific tasks.

Tasks Including:

- Reading input.
- Storing values in memory.
- Performing arithmetic operations.
- Outputting data.
- Each instruction has an Opcode that specifies the task to be performed.
- Operands/Parameters: Provide additional information needed to execute the instruction.

Instruction
Load
Store
Add
Subtract
Input
Output
End
Branch if zero
Branch if zero or positive
Branch always
Data storage

Mnemonic Code

Mnemonic Codes in LMC are human-readable representations of instructions.

Its purpose: Simplifies programming by using easy-to-remember abbreviations.

Ex:

- ADD: Adds the value from a specific memory location to the Accumulator.
- Other Mnemonic Codes: Used to represent various LMC instructions.

Mnemonic
LDA
STA
ADD
SUB
INP
OUT
HLT
BRZ
BRP
BRA
DAT

Machine Code

Machine Code: The binary code understood and executed by the computer.

LMC Instructions: Each instruction has a specific opcode representing its binary code.

Ex:

- LDA (Load) → Opcode 5
- ADD (Addition) → Opcode 1

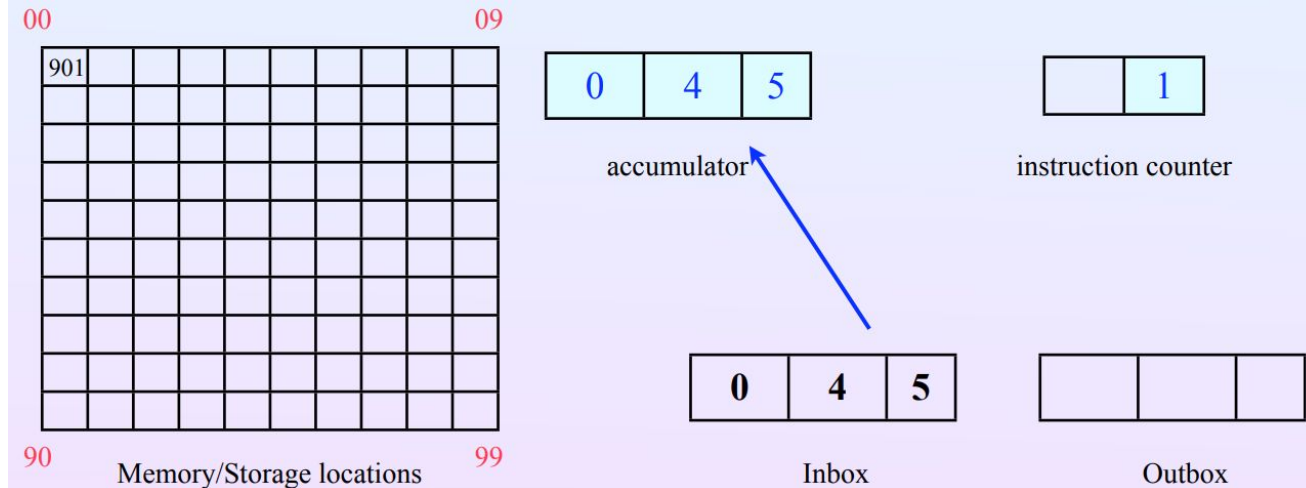
It is used for execution: Machine code directly controls the CPU's operations.

MachineCode
5xx
3xx
1xx
2xx
901
902
000
7xx
8xx
6xx

Input operation (INP)

The Input operation here - with 901 in location 00

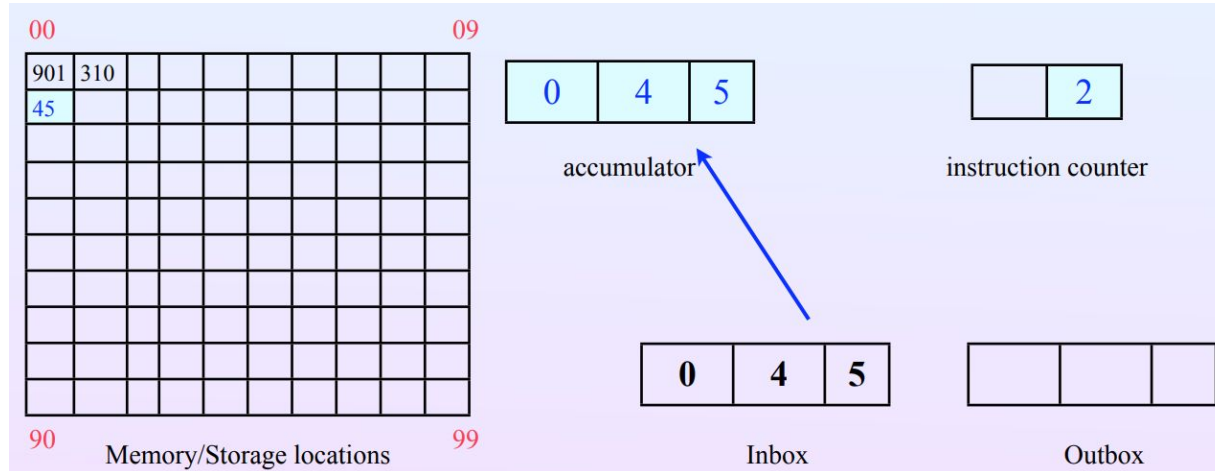
- takes 045 in the Inbox and places it in the accumulator
- increments the instruction counter from 0 to 1



Store operation (STA)

The Store operation here - with 310 in location 01

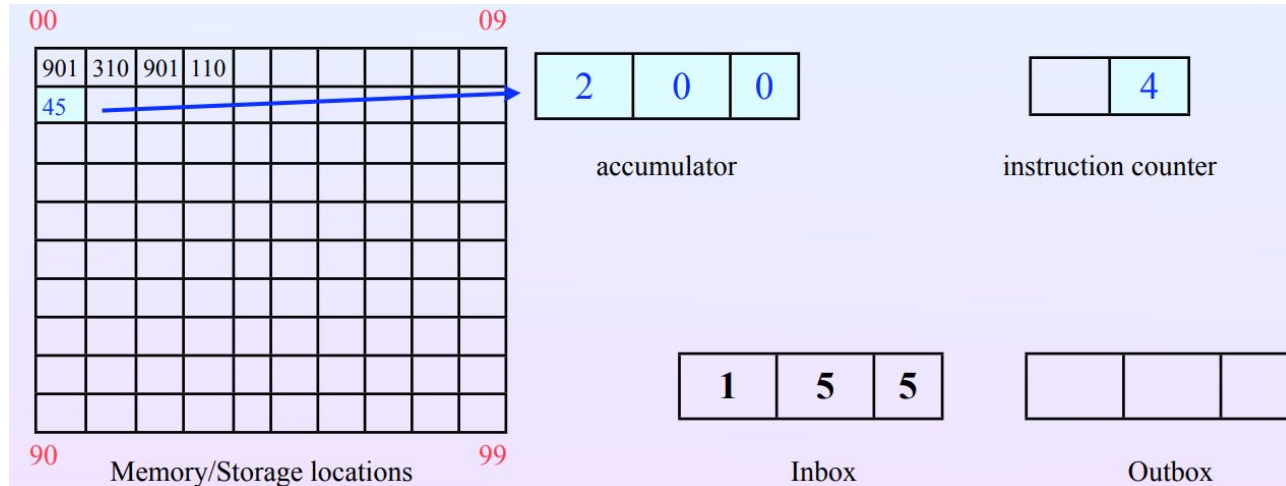
- takes 45 in the accumulator and places it in memory location 10
- increments the instruction counter from 1 to 2
- the accumulator retains its value



Add operation (ADD)

An add operation is performed with a 110 in memory location 03

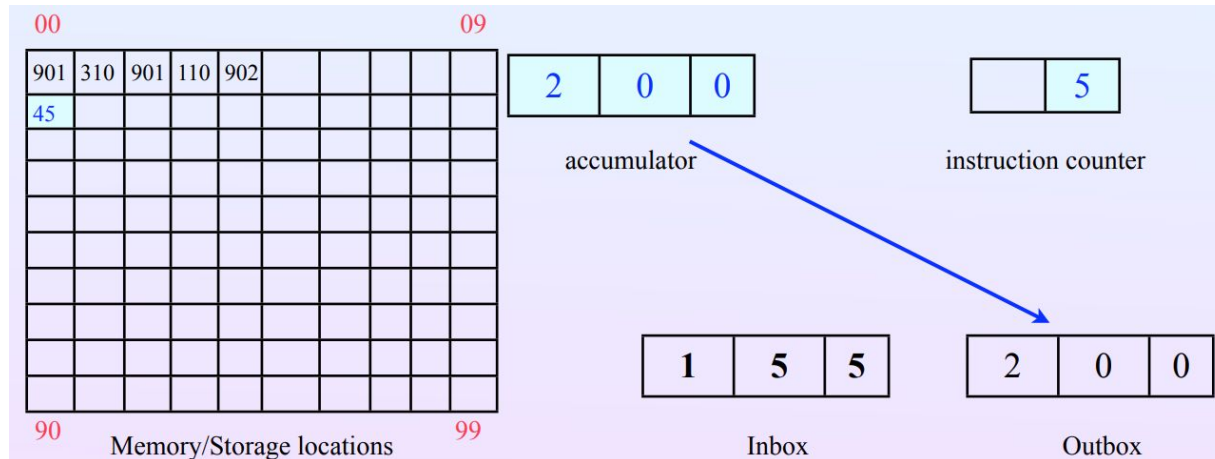
- the right 2 digits signify that the contents of location 10 are added to the accumulator giving value 200
- the instruction counter from 3 to 4



Out operation (OUT)

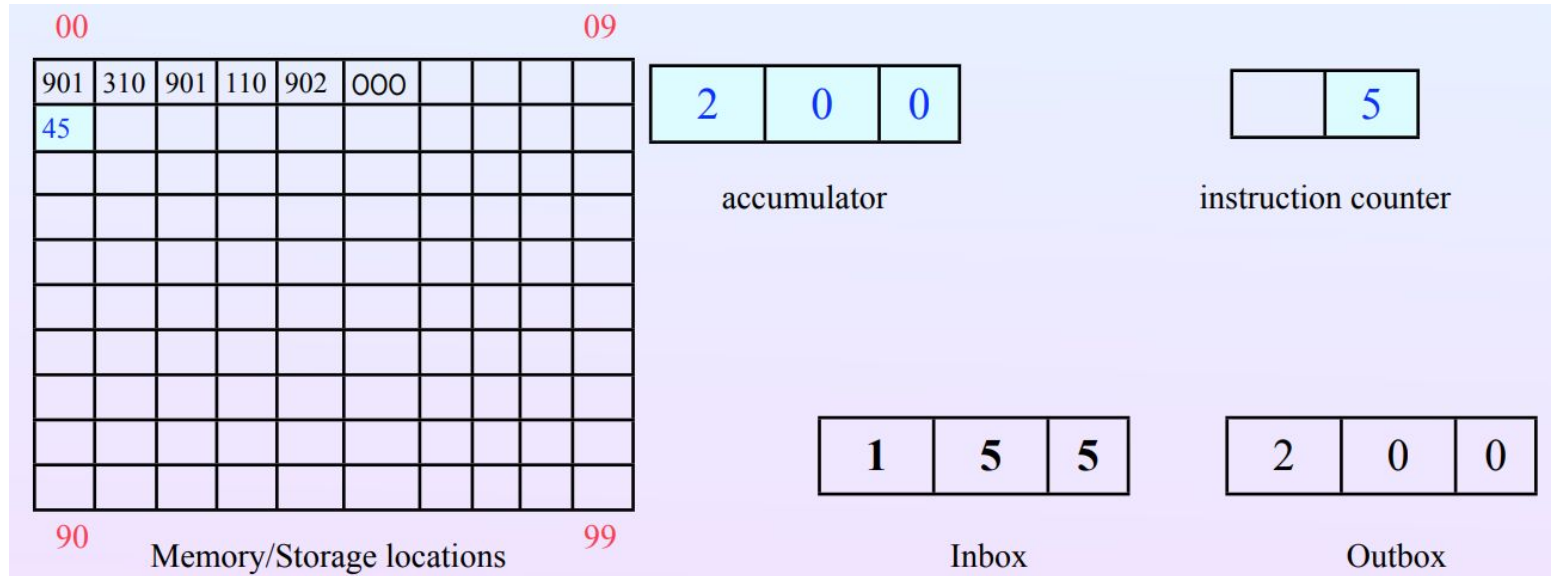
The Out operation is accomplished here with a 902 in location 05

- the contents accumulator is put in the outbox - contents of accumulator not changed
- the instruction counter is incremented from 4 to 5



HALT operation

The Halt operation is accomplished here with a 000 in location 06. It signifies the end of the program



The sequence of instructions constructed was:

901	310	910	110	902	000
-----	-----	-----	-----	-----	-----

It constitutes a Little Man program whose function adds two numbers. We call above the machine code of the Little Man program.

Corresponding assembly code is:

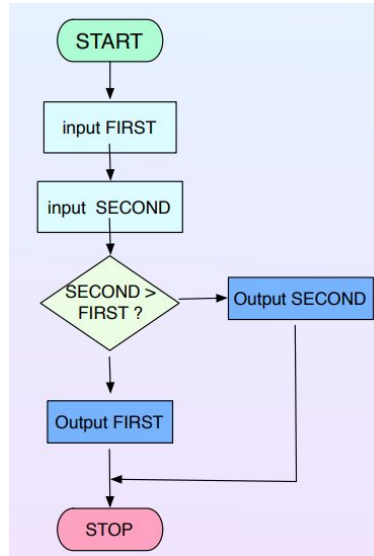
	INP	
	STA	FIRST
	INP	
	ADD	FIRST
	OUT	
	END	
FIRST	DAT	

Identifies FIRST as data - the compiler then allocates a memory position called FIRST. Our program has FIRST = 10. The “label” FIRST identifies a position in memory. It happens to be the 10th position.

Branching and Labels

Branching instructions allow program to execute some instructions under one condition and other instructions under other conditions

Example: diagram of decision process to determine greater of 2 numbers



To implement example in source code using Little Man mnemonics - there is notion of the label of an instruction to be branched to or the label of a data storage position

Assembly code:

LABEL	CODE	ARGUMENT	DESCRIPTION
	INP		input data = x to accumulator
	STA	FIRST	place accumulator data x in memory location defined by FIRST
	INP		input data = y to accumulator
	STA	SECOND	place accumulator data y in location defined by SECOND
	SUB	FIRST	subtract data in First from data in accumulator i.e. $y - x$
	BRP	SEC_BIG	if accumulator data $(y-x) \geq 0$, branch to SEC_BIG instruction
	LDA	FIRST	otherwise $x < y$, so load the value x in FIRST to accumulator
	OUT		output the value x
	BRA	END_PROG	go to the end instruction
SEC_BIG	LDA	SECOND	here $y \geq x$ - so load value y in SECOND to accumulator
	OUT		output the value y
END_PROG	HLT		end the program
FIRST	DAT		declare the data region labeled FIRST
SECOND	DAT		declare the data region labeled SECOND

6. Execution Cycle

LMC Execution Cycle repeats for each instruction in the program.

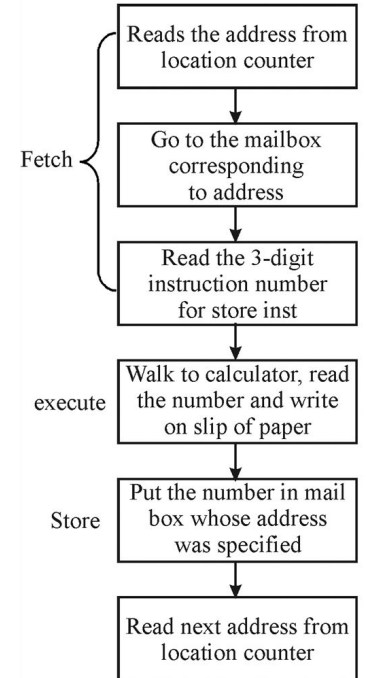
1. Fetch: Retrieves instruction from memory using the Program Counter.
2. Decode: Determines the operation to be performed.
3. Execute: Performs the instruction, updating the Accumulator or memory.
4. Update PC: Moves the Program Counter to the next instruction.

Sequential Execution: Instructions are executed one at a time, from first to last.

Branching & Jumping: Instructions can jump to different memory locations based on conditions:

- BRZ: Jumps if Accumulator = 0.
- BRP: Jumps if Accumulator > 0.

Little Man Computer's execution cycle



7. Memory

Memory in LMC stores both instructions and data for program execution. It has limited Capacity: Typically 100 memory locations.

Storage Type:

- Instructions → Stored as binary codes.
- Data values → Stored as decimal numbers.

Memory Address

Each memory location has a unique address (a number representing its position).

Used to access stored instructions and data during execution.

8. LIMITATIONS

1. Limited Instruction Set:

- a. Only includes basic arithmetic, memory operations, input/output, and control transfer.
- b. Difficult to write complex programs requiring advanced operations.

2. Small Memory Size:

- a. Typically 100 memory locations.
- b. Limits storage for large data sets and complex programs.

Overcoming the Limitations of the Little Man Computer (LMC)

- Using Subroutines:

Breaks large programs into smaller, reusable parts.

Helps manage complexity despite the limited instruction set.

- Using Labels & Branching:

Labels mark specific points in the program.

Branching (BRZ, BRP, BRA) allows jumping to different parts of the program based on conditions (e.g., Accumulator = 0 or positive).

- Using Assembly Language:

A low-level language closer to machine code.

Helps write more efficient programs within LMC's limitations.

9. Sample Program

This program prompts the user for a number, adds the value 5 to it, and outputs the result.

```
INP      # Read number from input device
STA NUM  # Store number in memory location labeled NUM
LDA NUM  # Load number from memory location labeled NUM into the Accumulator
ADD FIVE # Add 5 to the number
OUT      # Output the result to the output device
HLT      # Halt the program
NUM DAT  # Declare a memory location labeled NUM as a data value
FIVE DAT 5 # Declare a memory location labeled FIVE as a data value (with value 5)
```

(LMC use only direct addressing mode)

In-Class Activity

1. Why is the instruction cycle called a cycle?
2. Is assembly language easier or more difficult to translate than high-level languages? Justify your answer.
3. Are there any languages that are a lower-level than assembly?

Thank You

27. 03. 2026